

Self-HTR: A Novel Self-Supervised Handwritten Text Recognition Framework Using Generative Adversarial Networks

No Author Given

No Institute Given

Abstract. Robust handwritten text recognition (HTR) systems require diverse and abundant annotated data, which can be expensive to acquire. Unlike commonly used methods, such as transfer learning and data augmentation, that reduce the need for labeled data, this article proposes a novel self-supervised handwritten text recognition (HTR) framework to eliminate that need: self-HTR. Self-HTR leverages a high-level loss that informs about the textual dissimilarity between an input image and an image generated with a pretrained generative adversarial network containing the network’s predicted text. Based on preliminary experiments with handwritten character recognition, experiments were conducted on the IAM dataset for word recognition. Self-HTR was implemented by integrating the generative architecture GANwriting with a CNN-BLSTM architecture. Experiments were conducted with two image-based and two style-invariant losses. Trained on IAM images recreated with GANwriting and using transfer learning, self-HTR achieved a character- and word error rate of 0.44% and 1.56% on recreated test data. While on simulated data, these results indicate the potential of self-HTR on real data. Future research is necessary to validate the framework further on real data and use high-quality handwritten text generation models.

Keywords: Handwriting recognition · handwritten text recognition · handwritten text generation · self-supervised learning

1 Introduction

Handwritten text recognition (HTR), or the transcription of scanned handwritten documents to digital text, is a complex problem that is still actively researched. Since high-quality HTR systems need to handle different writing styles and languages, they require highly variable training data. As current state-of-the-art HTR systems rely on supervised deep learning, vast amounts of annotated data are necessary for their high performance. However, annotating documents, especially historical documents, takes much time and effort. At the same time, the demand for digital processing of textual information, as well as expectations of these systems, is increasing with the rapid developments of large language models.

Efforts have been made to deal with limited data availability, both in terms of number and variability. Transfer learning allows for the incorporation of knowledge from related tasks by using a pretrained neural network and fine-tuning its weights on the HTR task. This improves transcription quality and speeds up training times [3]. Another frequently used method is data augmentation, where the data variability is increased by obtaining new data through applying, e.g., morphological and geometrical operations to the available data. A more advanced technique that is applied for data augmentation is handwritten text generation (HTG), which aims to generate realistic images of handwriting in arbitrary writer styles and content. In contrast to transfer learning and traditional data augmentation, it can improve HTR systems’ ability to generalize to unseen textual content by explicitly increasing the linguistic variability of the training data and has the potential to replace real training images. However, this is not yet possible [24].

The current research proposes self-HTR, a novel framework where HTG is integrated with self-supervised learning to eliminate the need for annotated data during HTR model training. Inspired by TextStyleBrush [20], self-HTR is trained with the dissimilarity between an input image and a generated image containing the HTR-predicted text as a loss function. It is based on the idea that handwriting in the same style and of the same text is more similar compared to handwriting in the same style and different text. The generated image is produced by a pretrained HTG model conditioned on writing style and textual content. An overview of the proposed self-HTR framework is shown in Figure 1.

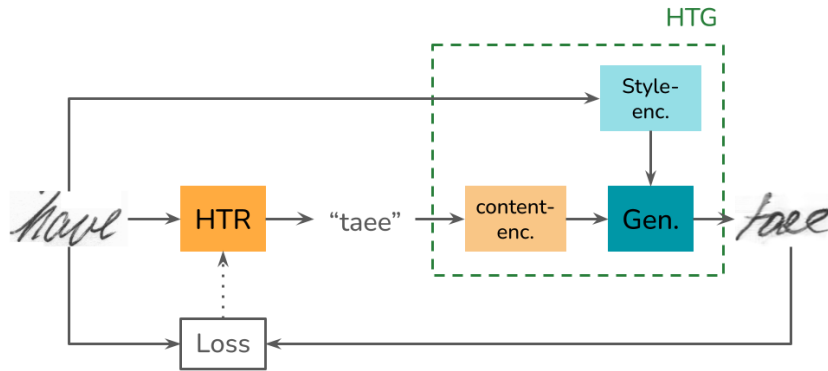


Fig. 1: The proposed self-supervised HTR framework. It uses a pretrained HTG model with style and content encoders to generate synthetic images with HTR-predicted text. The loss function computes the dissimilarity between the input and synthetic images.

The self-supervised framework was investigated with the IAM dataset for sequential word recognition, which is the standard level of abstraction in HTG. This was done after an initial exploration of the framework’s implementation and appropriate loss functions with handwritten character classification (HCR) on the MNIST dataset positively indicated its feasibility (see Supplementary Materials Section 5). Inspired by image-generation tasks and word spotting, we experimented with two image-based and two style-invariant loss functions. We show that self-HTR, as proposed and implemented in this research, can learn useful information if the input and generated images have minimal differences and the system has previous knowledge about the task. The implementation of the methodology presented in this paper is publicly available at <https://github.com/Lisa-dk/self-supervised-htr>.

2 Related Works

2.1 Handwritten Text Recognition

HTR models primarily use either recurrent neural networks (RNNs) or transformer architectures [21, 31]. RNN-based models typically use bi-directional long short-term memory (BLSTM) cells to handle long-term time dependencies while mitigating exploding and vanishing gradient issues. The addition of the CTC framework [10] enables end-to-end training without explicit alignment between the images and labels. Initially, these models processed raw image pixels to multi-dimensional BLSTMs [11], but were computationally expensive. Convolutional neural networks were thus applied before BLSTMs (CNN-BLSTM) for more efficient feature extraction [27]. Another approach is to use gates in the CNN to determine a feature’s relevance in the input image [6]. HTR-Flor, an adaptation to this approach, achieved competitive performance with the state-of-the-art while maintaining low computational costs [31].

One weakness of RNN-based HTR models is their reliance on statistical language models in post-processing. The latter allows for better generalization to out-of-vocabulary (OOV) words, never seen in training, which HTR models typically struggle with. Ideally, HTR models have inherent language modeling capabilities. This is addressed in attention-based approaches, which have been shown to be effective in recent years. These were initially sequence-to-sequence encoder-decoder models with an integrated attention mechanism for label-image alignment [23, 26]. With the successful application of the transformer architecture to natural language processing, HTR models adopted this architecture as well, with trOCR currently being the state-of-the-art model [21]. However, transformer-based models require large labeled datasets and are computationally expensive. For example, HTR-Flor has 0.8 million parameters, whereas the smallest TrOCR model has 64 million parameters [21].

2.2 Handwritten Text Generation

Handwritten text generation (HTG) is mainly used to create labeled images in arbitrary content and style. Current HTG models rely on generative adversarial

networks (GANs) or diffusion models. A standard GAN consists of a discriminator that identifies real and synthetic images and a generator model that synthesizes images [9]. Their application to HTG was introduced by [2], who conditioned the GAN on content through BLSTM-encoded text embeddings and adding a CTC loss on HTR predictions of the generated images to the generator loss for enforcing the text content. Later, ScrabbleGAN was proposed, which generated images of unconstrained lengths with arbitrary contents [7]. However, both approaches lacked variability in the synthetic images and could not mimic specific writers’ styles.

HTG GAN-based models adapted and extended [2] ScrabbleGAN by incorporating additional constraints to the generator loss. GANwriting, used in this study, is conditioned on writing style and adds a loss term from a writer classifier [15]. HiGAN and HiGAN+ [1, 8] enabled generating images containing arbitrary lengths of text, but increased computational costs and model complexity due to requiring as much as eight loss terms. Later transformer-based models, such as Handwriting Transformers [5] and the visual archetype transformer [25], further improved style imitation and OOV word generation.

Recently, diffusion models have gained attention in HTG research due to their high image quality. These models learn to reverse a process where sampled noise is repeatedly added to input data such that the data resembles Gaussian noise [30]. HTG models commonly use the Denoising Diffusion Probabilistic Model (DDPM) [13], with the error between the true and estimated noise as the loss. However, DDPMs operate directly on images and thus require high computational resources. Addressing this, [24] introduced WordStylist, which uses latent diffusion [29] for more efficient image generation, and showed state-of-the-art image quality.

3 Methods

3.1 Dataset

The IAM handwriting database [22] was used for the training and evaluation of self-HTR. It contains both annotated writing styles and texts, which are necessary for training and evaluating the HTG and HTR models. We used the training and validation partitions as provided by [15], following HTG literature. The partitions are mutually exclusive with regard to writing styles. The dataset contains word images of 2–7 characters with punctuation and digits removed. It contains 44,421 training samples from 339 writers and 18,437 validation samples from 161 writers (see Figure 2). To ensure independence between the HTR and HTG models, we assigned 50% of samples per writer in the IAM partition to either IAM-GEN (for the HTG model) or IAM-HTR (for the HTR model). The IAM-GEN partitions remained mutually exclusive w.r.t. writing styles, and those of IAM-HTR were refactored. Figure 3 provides an overview of these datasets.

As existing literature suggests that HTR systems struggle with OOV words, we split IAM-HTR into in-vocabulary (IAM-HTR-IV) and out-of-vocabulary

(IAM-HTR-OOV) datasets, each with dedicated training, validation, and test sets. IAM-HTR-OOV¹ (3,354 samples) contained words with 3–6 occurrences, ensuring at least one per partition, with splits of 2/1/1 for four samples, 2/2/1 for five samples, and 2/2/2 for six samples, respectively. IAM-HTR-IV (28,206 samples) included all remaining words, and 70%, 15%, and 15% were assigned to training, validation, and test sets, stratifying based on writing styles. One writing style was omitted from the validation set due to its low sample count.

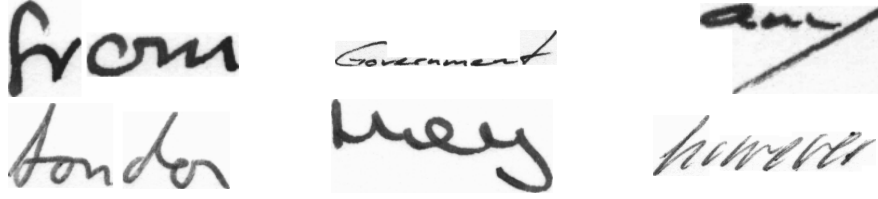


Fig. 2: Samples of the IAM database.

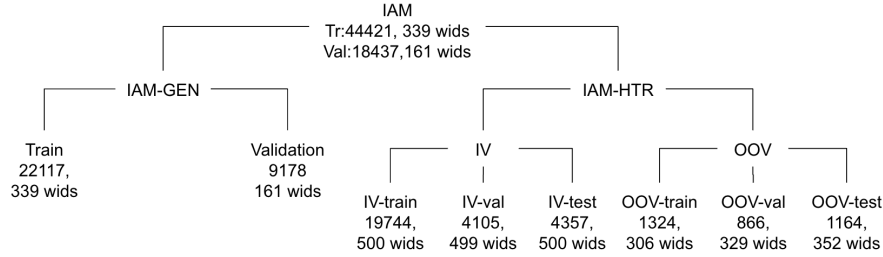


Fig. 3: The IAM-GEN and IAM-HTR partitions, each containing 50% of the IAM dataset as in [15]. The IAM-GEN partitions are writer-independent.

3.2 Image Preprocessing

To ensure compatibility and minimize computational costs in self-HTR, identical preprocessing was applied to the HTR and HTG pipelines. As generative models are sensitive to preprocessing, we adopted the image preprocessing steps of GANwriting [15], the selected HTG model. GANwriting operates on 64×216 grayscale images with pixel values in $[-1, 1]$. The variably-sized word images were hence first resized. This step was adapted in GANwriting’s pipeline to maintain

¹ This study only uses the IAM-HTR-OOV validation and test sets purposed for HTR model evaluation. See [17] for experiments involving all IAM-HTR-OOV partitions

the images' aspect ratios by padding images with the most frequent pixel value, following [31]. Additionally, images with small heights were padded to reduce scaling differences (see Figure 4). This adaptation was necessary as GANwriting resized based on height and cropped excess width, leading to information loss and incorrect labels. After resizing, images were normalized, inverted, and standardized using a mean and standard deviation of 0.5.

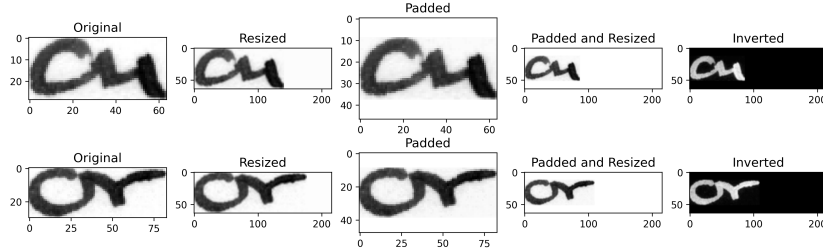


Fig. 4: Examples of how extra padding to the images' heights reduced scaling differences between word images.

3.3 Label Encoding

To incorporate the HTG generator in self-HTR, it needs to produce images based on the HTR networks' output. Following a successful implementation for self-supervised HCR, the HTR-predicted class probabilities were directly fed into the HTG generator. This incorporates HTR confidence and reduces computational costs. Hence, the generator was conditioned on one-hot encoded labels, which were aligned with HTR predictions by using a universal vocabulary. This vocabulary contained upper and lower case alphabetical characters (26 each), along with start, end, padding, and an unknown character token, totaling 56 characters. The labels were encoded by adding the start and end tokens, mapping characters to their indices in the vocabulary, adding padding tokens, and converting them to one-hot vectors. The implementation was adapted from [31].

3.4 Model Architecture

Handwritten Text Recognition Self-HTR is computationally expensive as it consists of two deep learning models and an image-based loss function, making a lightweight model ideal. However, state-of-the-art HTR models are implemented in TensorFlow, whereas GANwriting and other HTG models use PyTorch. To maintain compatibility and avoid the complexities of re-implementing GANwriting, we selected and implemented the simpler Puigcerver model [27]. This is a CNN-BLSTM and consists of five convolutional blocks for feature extraction and five recurrent blocks for sequence processing (see Figure 5). Since the

CNN-BLSTM is designed for line-text recognition, separate experiments were conducted to optimize it for word recognition (see Supplementary Materials, Section 1). Based hereon, column-wise concatenation of the CNN outputs was replaced with column-wise max pooling, enhancing translation invariance and reducing computational costs [28]. For further details, we refer to [27].

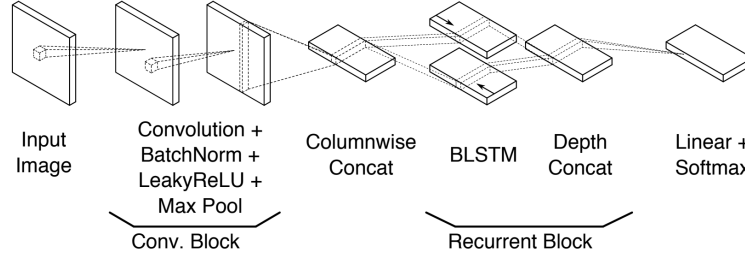


Fig. 5: Overview of the Puigcerver HTR model architecture. Figure obtained from [27].

Handwritten Text Generation GANwriting [15] was selected for text generation due to its relatively simple architecture and lower computational cost compared to other HTG models. The model consists of a generator H , a discriminator D , a writer classifier W , and a text recognizer R , which enforce realism, writer style, and text legibility, respectively (Figure 6). As only the pre-trained generator is used in self-HTR, we provide a brief overview of this model architecture. For further details, we refer to [15].

GANwriting’s generator synthesizes word images by conditioning on both style and content. A style encoder S extracts a latent representation $\hat{F}_s$ from a set of K style images of the same writer. The content encoder C maps each character c in the text input t to a one-dimensional character representation with an embedding layer. This is then mapped to character-wise (F_c) and global (f_c) representations. To integrate GANwriting into self-HTR, we replaced the embedding layer with one-hot encoded text inputs. The conditional generator G then combines the style and content representations to produce word images.

GANwriting was trained end-to-end, following [15]. Models R , W , D , and H were optimized separately in each epoch. The text recognizer and writer classifier were trained on real images using the Kullback-Leibler (KL) divergence and cross-entropy (CE) loss, respectively. The discriminator was trained with the adversarial loss, while the generator minimized the combined losses of all three auxiliary networks on the synthetic images and conditioned inputs. Training was performed with the Adam optimizer, using learning rates of 10^{-4} for D and H and 10^{-5} for R and W . Style representations were extracted from $K = 25$ images to account for IAM-GEN’s reduced training samples.

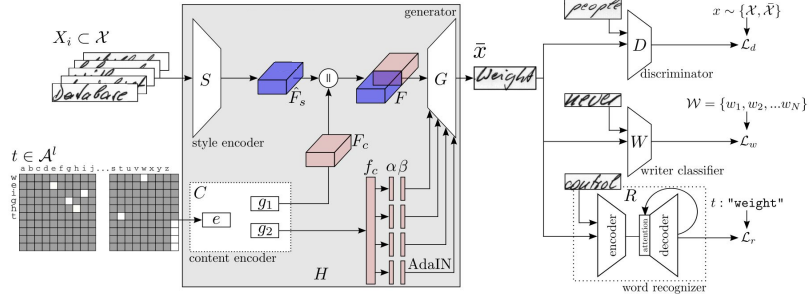


Fig. 6: An overview of GANwriting, adapted from [15]. Here, $\mathcal{X}$ is the set of word images, $\mathcal{W}$ is the set of writer IDs, and $\mathcal{A}$ is the vocabulary. The generator is conditioned on extracted style and content representations, while the discriminator, writer classifier, and text recognizer enforce realism, style, and legibility in the synthetic images.

Self-Supervised Handwritten Text Recognition The aforementioned HTR and HTG model architectures were integrated into the self-HTR framework by directly feeding the HTR-predicted class probabilities to the pretrained GANwriting generator and computing an image-based loss between the synthetic and input images. This required consistent image preprocessing and text label encoding across both self-HTR and HTG pipelines. Moreover, GANwriting’s generator was conditioned on one-hot encoded text labels. This minimized the frameworks’ computational costs and maintained the differentiability of the loss. Since the focus was on content, the writing style needed to remain unchanged. To enforce this, writer labels were used to randomly sample K word images (with replacement) from the same writer.

3.5 Loss Functions

Initial experiments in image-based self-supervised HCR showed that high-level loss functions were crucial for good model performance. Therefore, only such losses were considered for self-HTR. We experimented with the perceptual loss and the Structural Similarity Index Measure (SSIM) but found that writing style variations between real and synthetic images were detrimental to training. To address this, we propose two style-invariant losses: a Siamese Network loss and a text-based content loss.

Perceptual Loss [14] This loss function compares pixels of feature maps’ activations with C channels of height H and width W extracted with the j th layer of the VGG-16 network pretrained on ImageNet. Following the experiments for self-supervised HCR, the activations of feature maps were extracted at the 7th convolution. The perceptual loss is given by Equation 1, where ϕ is the pretrained VGG-16 network, y is the input image, and $\hat{y}$ is the synthetic image.

$$L_p(\hat{y}, y) = \frac{1}{C_j H_j W_j} \|\Phi_j(\hat{y}) - \Phi_j(y)\|_2^2 \quad (1)$$

Structural Similarity Index Measure [32] SSIM evaluates images based on luminance, contrast, and structure. The SSIM is computed per pixel with a sliding symmetric Gaussian window. Let p and $\hat{p}$ be pixels in a pair of input and synthetic images at the center of the sliding window covering P pixels. The SSIM is then computed as in Equation 2, where μ and σ represent the image patches' mean and standard deviations, and $\sigma_{\hat{p}p}$ is the image patches' covariance. C_1 and C_2 are constants. The SSIM ranges from -1 to 1, where 1 indicates perfectly similar images. The SSIM loss is then given by Equation 3.

$$SSIM(p, \hat{p}) = \frac{(2\mu_{\hat{p}}\mu_p + C_1)(2\sigma_{\hat{p}p} + C_2)}{(\mu_{\hat{p}}^2 + \mu_p^2 + C_2)(\sigma_{\hat{p}}^2 + \sigma_p^2 + C_2)} \quad (2)$$

$$L_{SSIM}(y, \hat{y}) = 1 - \frac{1}{P} \sum_{i=1}^P SSIM(y_i, \hat{y}_i) \quad (3)$$

Siamese Network Loss A Siamese Network maps pairs of input images to a latent space s.t. a distance metric is small for same-class images, and large otherwise. Inspired by its application to word spotting [4], we use a Siamese Network in a loss function to circumvent the stylistic differences between real and synthetic word images. The loss function is defined in Equation 4, where ϕ is a pretrained Siamese Network that extracts feature representations that have maximal similarity for same-content word images

$$L_{sia} = \|\phi(\hat{y}) - \phi(y)\|_2^2 \quad (4)$$

The Siamese Network was trained on a refactored IAM-GEN dataset (IAM-GEN-SIA) that contains all writer styles in its training, validation, and test sets. For each original image in this dataset, two similar and two dissimilar real and synthetic image pairs were generated with GANwriting's trained generator. The Siamese Network was trained with the contrastive loss and evaluated with the Euclidean distance between the similar and dissimilar image pairs. A two-tailed t-test confirmed that similar pairs had significantly smaller distances than dissimilar pairs ($t(2282) = 161.024$, $p < 0.005$; similar: $\mu = 0.107$, $\sigma = 0.07$; dissimilar: $\mu = 0.539$, $\sigma = 0.057$), validating the Siamese Network's ability to distinguish content from style. The methodology and results of the Siamese Network are further detailed in Supplementary Materials Section 3.

Text-based Content Loss Since the Siamese Network loss only implicitly encodes text, we also propose a style-invariant content loss using a pretrained HTR model. It replaces ϕ in the Siamese loss with a pretrained HTR network. We trained the same HTR network architecture as used in self-HTR on IAM-GEN-SIA to ensure its independence from IAM-HTR. It was trained with the cross-entropy loss. Its performance on this dataset is shown in Table 1. While this loss provides a direct comparison of textual content, the pretrained HTR model acts as a bottleneck to self-HTR, limiting performance.

Table 1: Performance of the best-performing supervised HTR model trained and evaluated on the original, real, images of IAM-GEN-SIA.

Training Data	Validation set		Test set	
	CER (%)	WER (%)	CER (%)	WER (%)
IAM-GEN-SIA	11.34	28.00	12.09	28.96

3.6 Experimental Setup

Self-HTR was trained on the IAM-HTR-IV dataset and evaluated on the IAM-HTR-IV and -OOV validation and test sets. To assess its effectiveness, self-HTR models were trained from scratch with He uniform weight initialization [12] and using the four aforementioned loss functions. Additionally, a ‘pretrained’ setting was explored, where the HTR network was initialized with the supervised HTR network trained on IAM-GEN-SIA. This supervised model is the same as was used in the text-based content loss computation (see section 3.5). This allowed us to investigate whether self-supervision could improve existing HTR models. All self-HTR models used the Adam optimizer, with an initial learning rate of 0.001 for training from scratch and 0.0001 for pretrained models, resulting in eight experiments.

Since self-HTR in initial experiments showed no indication of learning, additional models were trained on IAM-HTR recreated with GANwriting’s trained generator (recreated IAM-HTR). This reduced stylistic differences between input and synthetic images, isolating the effect of style mismatch. Since this focuses on writer style, these self-HTR models were trained only with the perceptual and SSIM losses. They used the Adam optimizer, with an initial learning rate of 0.001 for training from scratch and 0.0003 for pretrained models, resulting in eight experiments.

The pretrained generator used in the above experiments was trained for 3000 epochs, as it resulted in the best image quality compared to models in previous epochs, measured with the Fréchet inception distance (see Supplementary Materials Section 2). All models used a batch size of 16. Learning rates were reduced by a factor of 0.5 if the IAM-HTR-IV validation loss did not improve for 10 epochs. Moreover, early stopping with a patience of 15 epochs was applied to prevent overfitting.

3.7 Evaluation

Self-HTR models were evaluated and trained with IAM-HTR using character error rate (CER) and word error rate (WER). Models were tuned using the IV and OOV validation sets and tested on the respective test sets. Loss curves were analyzed to assess learning progress, and synthetic images with HTR-predicted text were examined for qualitative evaluation of self-HTR. For completeness, a

comparison of the supervised HTR architecture trained on IAM-HTR with state-of-the-art models is included. Additionally, this supervised model is compared with the best-performing self-HTR model.

4 Results

The supervised HTR model was trained on IAM-HTR and tested on its IV and OOV test sets. Comparison of its performance with state-of-the-art results reported on the word-level IAM dataset in literature is shown in Table 2. While the current supervised HTR architecture performance is not exceptionally high, it has to be considered that our goal was to evaluate the feasibility of self-HTR rather than surpass state-of-the-art results. Additionally, the model was trained on only half the IAM dataset, and other studies reported results on different test partitions, therefore serving only as to put our trained model’s performance into context.

Table 2: Word-level HTR results in the literature on the IAM database.

Method	CER (%)	WER (%)
Krishnan et. al., (2016) [19]	3.72	6.69
Krishnan et. al., (2023) [18]	1.67	3.62
Nikolaïou et. al., (2023) [24]	4.86	14.11
Kang et. al., (2019) [16]	6.88	17.45
Puigcerver et. al., (2017) [27], adapted	14.71	30.89

Self-HTR models were trained on IAM-HTR with each of the SSIM, perceptual loss, Siamese Network loss, and text-based content loss functions in both from scratch and pretrained settings. Results on the real IAM-HTR IV and OOV validation sets are shown in Table 3, along with the results of the pretrained HTR model for comparison. The models’ loss curves are included in the Supplementary Materials Section 4. Considering the from scratch setting, the high CER and WER showed that any learning that occurred for each of the loss functions was ineffective for the HTR task. In the pretrained setting, catastrophic forgetting is suggested by the substantial increase in CER and WER compared to that of the pretrained HTR model for each of the loss functions.

A better understanding of these results can be obtained by observing the models’ behaviors in the predicted synthetic images. In the from scratch setting (Figure 7), it can be observed that mode collapse-like behavior occurred for each of the four loss functions. In the images in the pretrained setting (Figure 8), this can also be observed for the SSIM and perceptual loss functions. For the Siamese Network loss, the characters are relatively legible but incorrect. Likely, this is due to the implicit content encoding in this loss function. The synthetic images

for the text-based content loss resulting from the pretrained setting are relatively coherent. It could be that catastrophic forgetting occurred for this model due to a mismatch in predictions by the pretrained HTR network between the real and synthetic images.

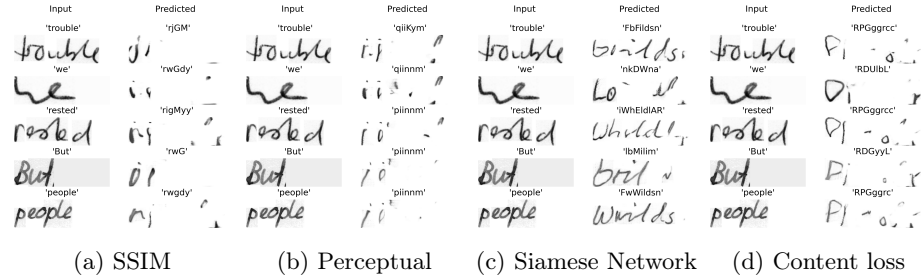


Fig. 7: Input images and their corresponding synthetic images with the predicted text for the SSIM (a), perceptual (b), Siamese Network (c), and text-based content (d) losses trained from scratch.

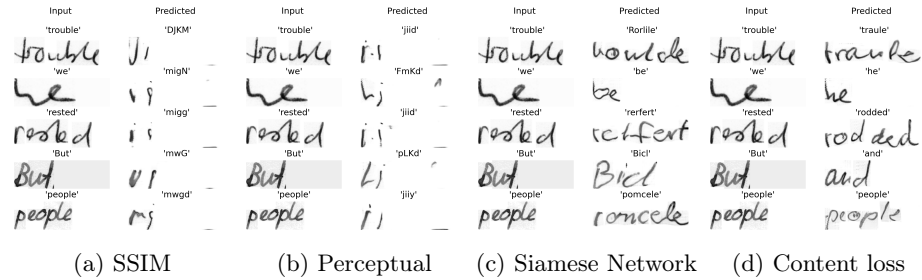


Fig. 8: Input images and their corresponding synthetic images with the predicted text for the SSIM (a), perceptual (b), Siamese Network (c), and text-based content (d) losses from the pretrained supervised HTR model.

The difference in model performance and behavior between the image-based and style-invariant losses suggests that the high error rates may be due to stylistic differences between real and synthetic images. To investigate this, we consider the results of self-HTR trained with the image-based losses on recreated IAM-HTR data shown in Table 4. These show that while learning remained limited from scratch, in the pretrained setting, the CER and WER considerably improved on both IV and OOV validation sets compared to the pretrained supervised HTR model, for both SSIM and perceptual loss functions. These results are also reflected in the predicted synthetic images, as seen in Figure 9.

Table 3: Self-supervised HTR results on the original IAM-HTR validation sets, for image and text-based loss functions.

Loss Function	Pre trained	IV Validation		OOV Validation	
		CER (%)	WER (%)	CER (%)	WER (%)
SSIM	No	96.33	100	94.14	100
	Yes	95.64	100	95.06	100
Perceptual loss	No	93.63	100	93.68	100
	Yes	94.49	100	94.22	100
Siamese Network loss	No	94.03	100	93.87	100
	Yes	59.89	81.98	74.94	99.89
Content-based loss	No	97.07	100	96.10	100
	Yes	21.26	48.50	39.66	88.86
Cross-entropy loss (Pretrained HTR)	-	10.89	26.38	24.39	63.39

As the perceptual loss resulted in the best-performing self-HTR model on recreated data, it was applied to the original IAM-HTR test data to analyze its generalization. Additionally, we compare it to the supervised HTR model trained on original IAM-HTR data. Table 5 shows that self-HTR’s error rates considerably increased on the original IAM-HTR-IV and OOV test data, indicating limited generalization to original data. This is likely due to limitations imposed by the synthetic image quality. Compared to the supervised HTR model, self-HTR showed a decreased performance on the IV test data, but an increased performance on the OOV test data. This may be due to some words of the IAM-HTR-OOV dataset overlapping with the IAM-GEN-SIA dataset that was used for pretraining. Overall, the supervised model showed a superior performance compared to self-HTR.

Table 4: Image-based self-supervised HTR results on the recreated IAM-HTR validation datasets.

Loss function	Pre- trained	IV Validation		OOV Validation	
		CER (%)	WER (%)	CER (%)	WER (%)
SSIM	No	66.79	85.12	87.99	100
	Yes	0.70	2.71	2.28	9.32
Perceptual	No	66.21	84.12	86.94	100
	Yes	0.36	1.45	1.46	6.48
CE (pretrained HTR)	-	5.60	17.05	14.84	49.65

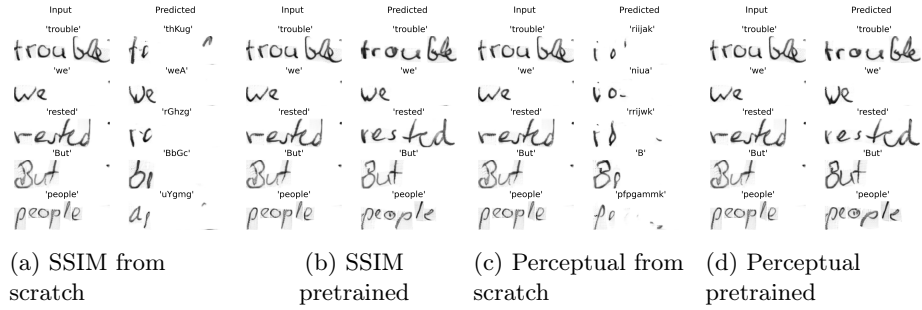


Fig. 9: Input images and their corresponding synthetic images with the predicted text for the SSIM and perceptual loss trained on recreated IAM-HTR from scratch (a), (c) and from the pretrained supervised HTR model (b), (d), respectively.

Table 5: Results on original and recreated IAM-HTR test data of the best-performing supervised and image-based self-supervised HTR models, trained on original and recreated IAM-HTR data, respectively. The self-HTR model was trained with the perceptual loss in the pretrained setting.

Test Data	Method	IV Test		OOV Test	
		CER (%)	WER (%)	CER (%)	WER (%)
Original IAM-HTR	Self-supervised	36.08	66.40	43.49	87.80
	Supervised	14.71	30.89	44.67	93.13
Recreated IAM-HTR	Self-supervised	0.44	1.56	1.83	7.04

5 Discussion

Self-HTR, as presented in this study, was only effective for improving existing HTR models with transfer learning and when synthetic images closely resemble real input images. Although self-HTR showed a limited performance, the framework can be improved in several ways. For one, the HTR network architecture has been shown to be sub-optimal compared to models such as HTR-Flor [31]. More crucial is self-HTR’s dependency on a high-quality HTG generator. Using a state-of-the-art generator such as WordStylist [24], which has improved style imitation and text legibility compared to GANwriting, could be more effective on real-life data and requires future investigation. Additionally, HTG models should be evaluated with a more appropriate image quality metric. This study used the FID, which is biased toward natural images as opposed to handwritten texts, and may thus have resulted in a sub-optimal model selection.

The loss function also greatly impacts the proposed framework’s performance. The image-based losses may have been too specific to capture high-level differences in character shape, explaining why models only performed well with transfer learning. Additional constraints, such as projection profiles, might allow the models to learn more relevant features. The features used in the perceptual loss, specifically, were likely sub-optimal as the VGG-16 network was pretrained on natural images instead of handwriting. A word classifier trained on diverse handwriting styles could be a better alternative, potentially replacing the Siamese Network loss as well by offering style-invariant content encoding.

While the above suggestions might improve self-HTR’s performance, its high computational costs remain a concern. Currently, it only shows good performances when improving an existing HTR model, and the pretrained HTR and HTG still need their own labeled training data. A more practical approach could involve using generative models and text-based similarity for post-processing tasks, such as automated verification or system monitoring, rather than for self-supervised learning. This would reduce dependence on HTR-HTG compatibility and offer greater flexibility in application.

6 Conclusions

This study proposed self-HTR: a novel framework that integrates HTR and HTG for self-supervised learning through an image-based loss function that measures the similarity between a synthetic image containing the HTR-predicted text and an input image w.r.t. the images’ textual contents. We showed that self-HTR would be effective for further improving existing HTR models only if minimal stylistic differences exist between the real and synthetic images. The framework’s potential for training HTR models without annotated labels should be more thoroughly investigated for real-life applications with higher-quality HTG models.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Higan: Handwriting imitation conditioned on arbitrary-length texts and disentangled styles **35**
2. Alonso, E., Moysset, B., Messina, R.: Adversarial generation of handwritten text images conditioned on sequences. In: 2019 International Conference on Document Analysis and Recognition (ICDAR). pp. 481–486 (2019)
3. Aradillas Jaramillo, J.C., Murillo-Fuentes, J.J., M. Olmos, P.: Boosting handwriting text recognition in small databases with transfer learning. In: 2018 16th International Conference on Frontiers in Handwriting Recognition (ICFHR). pp. 429–434 (2018). <https://doi.org/10.1109/ICFHR-2018.2018.00081>

4. Barakat, B.K., Alasam, R., El-Sana, J.: Word spotting using convolutional siamese network. In: 2018 13th IAPR International Workshop on Document Analysis Systems (DAS). pp. 229–234 (2018). <https://doi.org/10.1109/DAS.2018.67>
5. Bhunia, A., Khan, S., Cholakal, H., Anwer, R., Khan, F., Shah, M.: Handwriting transformers. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV). pp. 1066–1074. IEEE Computer Society, Los Alamitos, CA, USA (oct 2021)
6. Bluche, T., Messina, R.: Gated convolutional recurrent neural networks for multilingual handwriting recognition. In: 2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR). vol. 01, pp. 646–651 (2017)
7. Fogel, S., Averbuch-Elor, H., Cohen, S., Mazor, S., Litman, R.: Scrabblegan: Semi-supervised varying length handwritten text generation. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 4323–4332 (2020). <https://doi.org/10.1109/CVPR42600.2020.00438>
8. Gan, J., Wang, W., Leng, J., Gao, X.: HiGAN+: Handwriting imitation gan with disentangled representations. *ACM Trans. Graph.* **42**(1) (sep 2022)
9. Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., Bengio, Y.: Generative adversarial nets. In: Ghahramani, Z., Welling, M., Cortes, C., Lawrence, N., Weinberger, K. (eds.) *Advances in Neural Information Processing Systems*. vol. 27. Curran Associates, Inc. (2014)
10. Graves, A., Fernández, S., Gomez, F., Schmidhuber, J.: Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In: *Proceedings of the 23rd International Conference on Machine Learning*. p. 369–376. ICML '06, Association for Computing Machinery, New York, NY, USA (2006)
11. Graves, A., Schmidhuber, J.: Offline handwriting recognition with multidimensional recurrent neural networks. In: Koller, D., Schuurmans, D., Bengio, Y., Bottou, L. (eds.) *Advances in Neural Information Processing Systems*. vol. 21. Curran Associates, Inc. (2008)
12. He, K., Zhang, X., Ren, S., Sun, J.: Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In: 2015 IEEE International Conference on Computer Vision (ICCV). pp. 1026–1034 (2015)
13. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. In: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (eds.) *Advances in Neural Information Processing Systems*. vol. 33, pp. 6840–6851. Curran Associates, Inc. (2020)
14. Johnson, J., Alahi, A., Fei-Fei, L.: Perceptual losses for real-time style transfer and super-resolution (2016)
15. Kang, L., Riba, P., Wang, Y., Rusiñol, M., Fornés, A., Villegas, M.: GANwriting: Content-conditioned generation of styled handwritten word images. In: Vedaldi, A., Bischof, H., Brox, T., Frahm, J.M. (eds.) *Computer Vision – ECCV 2020*. pp. 273–289. Springer International Publishing, Cham (2020)
16. Kang, L., Toledo, J.I., Riba, P., Villegas, M., Fornés, A., Rusiñol, M.: Convoive, attend and spell: An attention-based sequence-to-sequence model for handwritten word recognition. In: Brox, T., Bruhn, A., Fritz, M. (eds.) *Pattern Recognition*. pp. 459–472. Springer International Publishing, Cham (2019)
17. Koopmans, L.: Towards self-supervised handwritten text recognition using generative adversarial networks (July 2024), available at <https://fse.studenttheses.ub.rug.nl/33045/>
18. krishnan, P., Dutta, K., Jawahar, C.V.: HwNET v3: a joint embedding framework for recognition and retrieval of handwritten text. *International Journal on Document Analysis and Recognition (IJDAR)* **26**(4), 401–417 (dec 2023)

19. Krishnan, P., Dutta, K., Jawahar, C.: Deep feature embedding for accurate recognition and retrieval of handwritten text. In: 2016 15th International Conference on Frontiers in Handwriting Recognition (ICFHR). pp. 289–294 (2016)
20. Krishnan, P., Kovvuri, R., Pang, G., Vassilev, B., Hassner, T.: TextStyleBrush: Transfer of text aesthetics from a single example. *IEEE Trans. Pattern Anal. Mach. Intell.* **45**(7), 9122–9134 (jul 2023)
21. Li, M., Lv, T., Chen, J., Cui, L., Lu, Y., Florencio, D., Zhang, C., Li, Z., Wei, F.: TrOCR: Transformer-based optical character recognition with pre-trained models. *arXiv preprint arXiv:2109.10282* (2022)
22. Marti, U., Bunke, H.: The IAM-database: an English sentence database for offline handwriting recognition. *International Journal on Document Analysis and Recognition* **5**, 39–46 (nov 2002)
23. Michael, J., Labahn, R., Gruning, T., Zollner, J.: Evaluating sequence-to-sequence models for handwritten text recognition. In: 2019 International Conference on Document Analysis and Recognition (ICDAR). pp. 1286–1293. IEEE Computer Society, Los Alamitos, CA, USA (sep 2019)
24. Nikolaidou, K., Retsinas, G., Christlein, V., Seuret, M., Sfikas, G., Smith, E.B., Mokayed, H., Liwicki, M.: WordStylist: styled verbatim handwritten text generation with latent diffusion models. In: International Conference on Document Analysis and Recognition. pp. 384–401. Springer (2023)
25. Pippi, V., Cascianelli, S., Cucchiara, R.: Handwritten text generation from visual archetypes. In: 2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 22458–22467. IEEE Computer Society, Los Alamitos, CA, USA (jun 2023)
26. Poulos, J., Valle, R.: Character-based handwritten text transcription with attention networks. *Neural Computing and Applications* **33**, 0563–10573 (aug 2021)
27. Puigcerver, J.: Are multidimensional recurrent layers really necessary for handwritten text recognition? In: 2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR). vol. 01, pp. 67–72 (2017)
28. Retsinas, G., Sfikas, G., Gatos, B., Nikou, C.: Best practices for a handwritten text recognition system. *arXiv preprint arXiv:2404.11339* (2024)
29. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B.: High-resolution image synthesis with latent diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 10684–10695 (June 2022)
30. Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., Ganguli, S.: Deep unsupervised learning using nonequilibrium thermodynamics. In: Bach, F., Blei, D. (eds.) Proceedings of the 32nd International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 37, pp. 2256–2265. PMLR, Lille, France (07–09 Jul 2015)
31. de Sousa Neto, A.F., Bezerra, B.L.D., Toselli, A.H., Lima, E.B.: HTR-Flor: A deep learning system for offline handwritten text recognition. In: 2020 33rd SIBGRAPI Conference on Graphics, Patterns and Images (SIBGRAPI). pp. 54–61 (2020). <https://doi.org/10.1109/SIBGRAPI51738.2020.00016>
32. Wang, Z., Bovik, A., Sheikh, H., Simoncelli, E.: Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* **13**(4), 600–612 (2004)